■ DKG BANK ENTERPRISE ATM

Technical Architecture & Interview Preparation Guide

Project Stack: *Django 5.0, DRF, SQLite (Local) / PostgreSQL (Production), Nginx, Docker, GitHub Actions*
Interview Core Focus: *Concurrency, Row-level Locking, API Design, Dockerization, Production Deployment on Render*
Author: *Antigravity Assistant & Technical Team*
Date: *June 2026 (Updated for Render Hosting Configuration)*

1. System Architecture Overview

DKG Bank is an enterprise-grade full-stack ATM application. It represents a **decoupled Django and Vanilla JS** architecture built to simulate core banking transactions under heavy production scenarios. Below are the design principles of the application:

- **Decoupled API-First Design:** The frontend (HTML/CSS/JS templates) interacts with the Django backend purely via REST API endpoints implemented using Django REST Framework (DRF) APIViews. This separates layout rendering from transactional business logic.
- **CSRF and Authentication Stack:** Security is implemented at the API level. User account management relies on Django's built-in django.contrib.auth session mechanism. CSRF cookies are dynamically fetched and attached by the Vanilla JS frontend to all state-modifying requests (POST).
- **Concurrency Safeguard:** Uses Django's database transaction blocks (transaction.atomic) and row-level locks (select_for_update) to prevent double-withdrawal race conditions, mathematically ensuring account balances never go below zero.
- **Responsive Glassmorphism Design:** The layout uses custom raw CSS to deliver a premium modern banking look featuring neon glows, translucid frosted cards, and subtle transitions.

System Flow Diagram

User Browser (Frontend)	**--> [HTTP Requests + CSRF Header] -->**	**Nginx Reverse Proxy**
Nginx Reverse Proxy	**--> [Port 80 to Port 8000] -->**	**Gunicorn (WSGI Server)**
Gunicorn (WSGI Server)	**--> [Spawns 4 worker threads] -->**	**Django Web Application**
Django Application	**--> [Row-Level Locking Query] -->**	**PostgreSQL / SQLite Database**

2. Project Directory Structure & Key Components

The project is structured with a root directory that contains configuration and deployment assets, and a main python source folder named DKG_ATM containing two separate Django apps:

Directory/File Path	Primary Role / Responsibilities
/build.sh & /DKG_ATM/build.sh	Render build scripts (installs packages, executes migrations, collectstatic).
/Procfile & /DKG_ATM/Procfile	Defines the WSGI startup command using Gunicorn server.
/DKG_ATM/DKG_ATM/	Project root configuration folder containing settings.py, urls.py, and wsgi.py.
/DKG_ATM/BankInterface/	The core banking backend app. Contains models.py, views.py (DRF APIs), serializers.py, and tests.py.
/DKG_ATM/UserInterface/	Frontend controller app. Manages HTML templates and login/registration routing.
/DKG_ATM/static/	Holds static files (CSS/JS) decoupled into UserInterface/ (auth pages) and BankInterface/ (dashboard).
/DKG_ATM/templets/	Stores the base HTML templates (index, register, next_page, history, etc.).
/DKG_ATM/requirements.txt	Lists python libraries (Django, DRF, whitenoise, reportlab, dj-database-url, gunicorn, etc.).
/DKG_ATM/nginx.conf	Production Nginx server configuration for reverse proxying and caching static files.

App Isolation & Clean Routing Design

Notice the complete division of labor: **UserInterface** is responsible for authentication views (login page rendering, forgot password templates, reset passwords, OTP requests) while **BankInterface** acts strictly as a transactions API (providing JSON endpoints for cash withdrawals, deposits, receipts, and balance checks). This follows the separation of concerns paradigm, facilitating maintenance and scalability.

3. Database Schema Design (BankInterface)

The application models represent bank accounts, transactions, and credit systems. It extends the default Django User model.

Account Model (Stores ATM profile information)

• user: OneToOneField to Django's built-in User. Cascades on deletion.
• card_number: Unique 16-character string representing the physical ATM card.
• card_pin: 4-character string checking verification.
• balance: DecimalField (max digits 12, decimal places 2) storing funds.
• is_locked: Boolean field. Automatically flips to True if user fails PIN checks 3 consecutive times.
• failed_attempts: Integer field incremented on bad PIN entries; resets to 0 on successful verification.
• customer_id: Unique 8-digit random string generated inside the save() override.
• account_number: Unique 12-digit random string generated inside the save() override.

Transaction Model (Stores immutable transaction records)

• account: ForeignKey linking directly to the Account model. Enables backwards queries.
• amount: Decimal field representing cash volume.
• transaction_type: Choices: Withdrawal, Deposit, Loan Disbursement, Loan Repayment, Transfer.
• timestamp: Auto-created timezone-aware date-time stamp marking the audit point.

Loan Model (Stores financial loan approvals)

• account: ForeignKey to the applicant account.
• loan_amount: Total requested loan principal.
• status: PENDING, APPROVED, REJECTED, CLOSED.

Code Insight: Custom ID Generators on Save

To guarantee unique identifiers without manual input, the Account.save() method is overridden:

```
def save(self, *args, **kwargs):
    if not self.account_number:
        while True:
            acc_num = ''.join(random.choices(string.digits, k=12))
            if not Account.objects.filter(account_number=acc_num).exists():
                self.account_number = acc_num
                break
    super().save(*args, **kwargs)
```

4. Decoupled REST APIs and Endpoints

The communication layer utilizes structured REST API requests. When a customer interacts with the ATM (e.g. keying a PIN or withdrawing cash), the client-side JavaScript issues a fetch() request with a JSON payload.

Endpoint	HTTP Verb	Payload Details	API Response Output
/verify_pin/	POST	{"card_pin": "1234"}	{"message": "Verified", "card_number": "...", ...}
/available_balance/	POST	{"card_number": "..."}	{"balance": 1000.00}
/withdraw/	POST	{"card_number": "...", "amount": 100}	{"message": "Withdrawal successful", ...}
/deposit/	POST	{"card_number": "...", "amount": 100}	{"message": "Deposit successful", ...}
/pin_change/	POST	{"card_number": "...", "new_pin": "5678"}	{"message": "PIN changed successfully"}
/loan_request/	POST	{"card_number": "...", "amount": 5000}	{"message": "Loan application submitted"}
/analytics/<card_number>/	GET	None (URL Param)	{"withdrawals": 500.00, "deposits": 1200.00}
/receipt/<tx_id>/	GET	None (URL Param)	PDF stream (Generated on-the-fly via ReportLab)

Security Safeguard: Dynamic CSRF Passing in JS

Because Django enforces Cross-Site Request Forgery protections, the Vanilla Javascript reads the token dynamically and injects it into the HTTP header of every request:

```
function getCSRFToken() {
    const input = document.querySelector('[name=csrfmiddlewaretoken]');
    return input ? input.value : getCookie('csrftoken') || '';
}

fetch('/withdraw/', {
    method: 'POST',
    headers: {
        'Content-Type': 'application/json',
        'X-CSRFToken': getCSRFToken()
    },
    body: JSON.stringify({ card_number: currentCardNumber, amount: amount })
})
```

5. Concurrency Control & Race Condition Prevention

Crucial Interview Topic: How do you prevent double-withdrawal race conditions? In high-traffic environments, two request threads could query a customer's account balance at the exact same millisecond. If both read a balance of $100 and attempt to withdraw $100 simultaneously, both threads might approve the operation before updating the database, causing the balance to drop to -$100 (a critical banking exploit).

The Row-Level Lock Solution: select_for_update

DKG Bank solves this by locking the database row during the transaction using Django's select_for_update() within a transaction block (transaction.atomic()):

```
with transaction.atomic():
    # Locks the account row. Any other concurrent query is blocked until this block completes.
    account = get_object_or_404(Account.objects.select_for_update(), card_number=card_number)

    if account.balance < amount:
        return Response({'error': 'Insufficient funds'}, status=status.HTTP_400_BAD_REQUEST)

    account.balance -= amount
    account.save()
    Transaction.objects.create(account=account, amount=amount, transaction_type='WITHDRAWAL')
```

How it works under the hood (SQL Translation)

Under PostgreSQL, this code generates the SQL query:
SELECT * FROM BankInterface_account WHERE card_number = '...' FOR UPDATE;
The FOR UPDATE clause tells the database engine to acquire an exclusive write lock on that row. If a second thread attempts to run a query with FOR UPDATE on the same row, PostgreSQL halts its execution and makes it wait in queue until the first thread's transaction commits or rolls back.

Threaded Concurrency Testing

The test suite includes a multi-threaded Python test (test_concurrent_withdrawals_prevent_negative_balance) that spawns concurrent threads using Python's concurrent.futures.ThreadPoolExecutor to intentionally hit the withdrawal API at the same time, confirming that one request succeeds while the second fails gracefully with an insufficient funds or lock error.

6. Enterprise Security Implementation

Security is implemented at several layers of the application to prevent automated attacks, brute-force hacking, and to provide a proper audit trail.

1. API Rate Limiting (Throttling)

To prevent script-kiddies and botnets from brute-forcing card PINs or spamming the OTP login endpoint, DKG Bank utilizes Django REST Framework's rate-limiting middleware configured in settings.py:

```
REST_FRAMEWORK = {
    'DEFAULT_THROTTLE_CLASSES': [
        'rest_framework.throttling.AnonRateThrottle',
        'rest_framework.throttling.UserRateThrottle'
    ],
    'DEFAULT_THROTTLE_RATES': {
        'anon': '20/min',   # 20 requests per minute max for guest calls
        'user': '100/min'   # 100 requests per minute max for logged users
    }
}
```

2. Immutable Audit Logging

Every crucial transactional operation (successful logins, deposits, withdrawals, card locking) is recorded to a secure, append-only file bank_audit.log using Python's logging module. If a security analyst wants to trace account behavior, the logs list timestamped events in the following verbose format:

INFO 2026-06-10 13:30:00 views Successful Withdrawal of $200 from card 1111222233334444. New Balance: $4800.00
CRITICAL 2026-06-10 13:31:00 views Account AUTOMATICALLY LOCKED for user guest_user after 3 failed attempts.

3. Session-Backed OTP Verification

When a user signs in using the OTP flow, a random 6-digit string is generated on the server and saved to the Django database-backed session (request.session['login_otp'] = otp) while being sent to the user's email via Gmail SMTP. When submitted, the backend verifies the match and clears the session variable to guarantee single-use. This prevents replay attacks.

7. Containerization & Production Docker Setup

To ensure development environments match production exactly, the application is containerized using Docker, Docker Compose, and Nginx. This setup isolates the Python runtime, PostgreSQL database, and web server proxy.

Dockerfile Analysis

The Dockerfile uses a slim base python image (python:3.12-slim) to keep image size small. It installs necessary build dependencies for PostgreSQL (libpq-dev, gcc), copies the code, collects static files, and creates a **non-root application user** (appuser) for security. The container is executed using Gunicorn with optimized workers (4 workers, sync worker class, 60s timeout).

Docker Compose Coordination (docker-compose.yml)

Docker Compose coordinates three service containers on a bridge network:

Container Service	Role in Stack	Ports / Volumes / Dependencies
db	PostgreSQL Database engine. Stores tables securely.	Port 5432. Persists data via postgres_data volume. Health checked.
web	The Django + Gunicorn application container.	Port 8000. Depends on db being healthy. Collects static and runs mi
nginx	Nginx web server acting as a reverse proxy.	Port 80. Maps static files directly; forwards dynamic requests to web

Production Nginx Reverse Proxy Config (nginx.conf)

Nginx serves static files (CSS, JS) directly from a shared Docker volume (/app/staticfiles/) with cache-control headers enabled for 30 days. This bypasses Django entirely for static assets, saving significant CPU resources. All other requests are forwarded to the upstream Gunicorn backend:

```
location /static/ {
    alias /app/staticfiles/;
    expires 30d;
    add_header Cache-Control "public, immutable";
}
location / {
    proxy_pass http://web;
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP $remote_addr;
}
```

8. CI/CD Pipeline & Production Deployment

For modern deployment, DKG Bank leverages automated continuous integration (GitHub Actions) and hosting configurations on Render.

1. GitHub Actions CI (django.yml)

Whenever code is pushed to master or main, GitHub Actions automatically executes the CI pipeline to verify code quality. Under a fresh Ubuntu runner environment, it:

• Spins up a temporary PostgreSQL service container.
• Checks out the source code and configures Python 3.11.
• Installs dependencies from requirements.txt along with DB drivers.
• Runs database migrations (python manage.py migrate).
• Runs the test suite (python manage.py test) to prevent regression bugs from reaching production.

2. Production Deployment on Render

Render connects to the GitHub repository and deploys the app dynamically using the newly implemented configuration files:

- **Unified Build Script (build.sh):** Render runs ./build.sh during the build phase. This script automates installing packages, running database migrations on the live database, collecting static files using WhiteNoise, and executing create_test_user.py to ensure a default test account is seeded.
- **WSGI Gunicorn Server (Procfile):** Informs Render to start Gunicorn using the proper wsgi module, allowing the app to handle production web traffic safely.
- **Database SSL Requirement:** Render's PostgreSQL databases require SSL. The database setup in settings.py is updated to detect the RENDER environment variable and dynamically enable ssl_require=True only in the production cloud database, keeping local sqlite running without SSL problems.
- **Email Fallback Interface:** If registration SMTP is offline, the API response is modified to return the newly generated card number and ATM PIN in the JSON payload, displaying it directly on the success modal on the screen. This guarantees zero lockouts during testing.

Summary of Render Deployment Settings

Setting Field	Value / Configuration	Purpose
Build Command	./build.sh	Runs requirements install, DB migration, static collect, ar
Start Command	gunicorn --chdir DKG_ATM DKG_ATM.wsgi --log-file -	Launches the production WSGI server instance.
Env: DATABASE_URL	postgres://...	Auto-injected by Render PostgreSQL connection binding
Env: RENDER	true	Used by settings.py to force database SSL connection m